

Web Servers and WSGI (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p4-web-servers-and-wsgi>  <https://github.com/learn-co-curriculum/python-p4-web-servers-and-wsgi/issues/new>

Learning Goals

- Describe the components of a web application framework.
-

Key Vocab

- **Web Framework:** software that is designed to support the development of web applications. Web frameworks provide built-in tools for generating web servers, turning Python objects into HTML, and more.
 - **Extension:** a package or module that adds functionality to a Flask application that it does not have by default.
 - **Request:** an attempt by one machine to contact another over the internet.
 - **Client:** an application or machine that accesses services being provided by a server through the internet.
 - **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.
 - **Web Server Gateway Interface (WSGI):** an interface between web servers and applications.
 - **Template Engine:** software that takes in strings with tokenized values, replacing the tokens with their values as output in a web browser.
-

Introduction

This lesson is a code along. Please fork and clone it before continuing!

Web servers are the software and hardware that allow users to access resources through the internet. While there are a whole range of jobs that are dedicated to creating and maintaining servers, building a basic server for yourself is actually very simple:

```
$ python -m http.server
# => Serving HTTP on :: port 8000 (http://[::]:8000/) ...
```

Navigate to `localhost:8000` in your browser and you should see the following:

Directory listing for /

- [.canvas](#)
 - [.git/](#)
 - [.gitignore](#)
 - [CONTRIBUTING.md](#)
 - [lib/](#)
 - [LICENSE.md](#)
 - [README.md](#)
 - [testing/](#)
-

`http.server` is a module in Python's standard library that creates a simple webpage that can respond to clients through an open URL and port. Run from the command line, it creates a navigable directory structure with URLs mimicking the path for each resource. Clicking on a directory shows its contents, and clicking on a file downloads it to your computer.

Ports

Notice the "8000" after your hostname when you run `http.server` ? This is the **port** that allows you to access the resources that your server is serving. Ports are a networking concept that you won't use a whole lot in full-stack web development, but you'll see them whenever you run a server.

You can think of ports like *extensions* on phone numbers. If you call your doctor's office, you're accessing the practice with the phone number itself (this is like the URL), but you're reaching the specific doctor through the extension.

Some ports are explicit and specific- you can't access your `http.server` resource without including `:8000` at the end of your URL. Others are implied by the protocol used to connect to the resource. HTTP is always `:80` , HTTPS is always `:443` , FTP (File Transfer Protocol) uses `:20` and `:21` , etc.

NOTE: If you see a lock next to the URL (or `https://`), it means that you're accessing a resource protected by Transport Layer Security, or TLS. Though it doesn't show `:443` , this is the port that you're using. You should always check that a website is protected before entering any personal information!

Running your own server, you can choose any port between 1024 and 65,535 to make your application accessible in the browser.

`http.server` defaults to 8000 as seen above, and Flask applications default to port 5000. We're typically going to change this to 5555 because of another application running on port 5000: Apple's Airplay!

NOTE: Some firewalls block certain ports. If you see a 403 response code in your browser, you probably need to change the port that your server is running on.

Web Server Gateway Interface (WSGI)

The Web Server Gateway Interface (usually called WSGI) is a specification that tells our Python code on a client or server how to communicate effectively over HTTP (or HTTPS, of course). WSGIs were introduced by PEP 333 (updated to PEP 3333 after the release of Python 3) because the web frameworks that existed at the time were not able to work with many popular servers without writing custom code. Developers usually have strong preferences about the frameworks and libraries that they implement applications with, so this limitation prevented many from making the switch from Java to Python.

The implementation of WSGI eliminated this concern. Because WSGIs could be configured to work with Python on one side to process requests and web servers on the other side to process responses, developers no longer had to worry about designing whole applications around their choice of server. WSGIs today are configured to work with most popular servers out of the box, and many even include development servers for you to work with as you build your application!

Werkzeug

Werkzeug  (<https://werkzeug.palletsprojects.com/en/2.2.x/quickstart/>) is the WSGI library that we will be using in Phase 4. Werkzeug was developed by the Armin Ronacher (the author of Flask!) and is maintained by the Pallets Projects team. It includes a number of features that will come in handy as we start to build our first Python web applications:

- An in-browser debugger.
- Robust classes for requests and responses.
- Routing, auto-generation and management of URLs.
- A development server.
- A testing framework that does not require a running server.

Let's take a look at a simple application that we can run with Werkzeug alone.

A Simple Werkzeug Application

Run `pipenv install && pipenv shell` to generate and enter your virtual environment. This will install Werkzeug, alongside our usual testing and debugging libraries.

In `server/werkzeug_app.py`, enter the following code:

```
from werkzeug.wrappers import Request, Response

@Request.application
def application(request):
    print(f'This web server is running at {request.remote_addr}')
    return Response('A WSGI generated this response!')

if __name__ == '__main__':
    from werkzeug.serving import run_simple
    run_simple(
        hostname='localhost',
        port=5555,
        application=application
    )
```

Let's break down our code a bit:

```
@Request.application
def application(request):
    print(f'This web server is running at {request.remote_addr}')
    return Response('A WSGI generated this response!')
```

This is the sole function inside of our script. (You can call it anything, we used `application` for simplicity's sake.) It is decorated with the `Request.application` method, which tells it to run any code inside of the function in the browser at the location we specify with our development server.

```
run_simple(
    hostname='localhost',
    port=5555,
    application=application
)
```

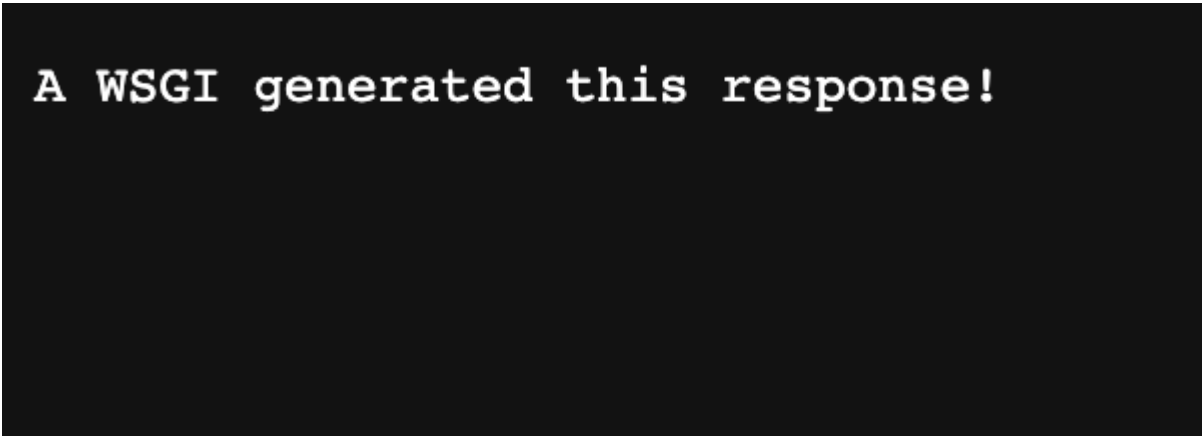
The `run_simple()` method runs a server for a one-page application without complications. It is not suited for a production server that supports millions of users, but it gives us the tools we need to develop new pages for the web applications that we eventually deploy to those servers.

`run_simple()` requires three arguments: a `hostname` (generally `localhost`, as it is typically used for local development), a `port`, and an `application`. This application will be defined in a function somewhere in the file- as we saw before, we named ours `application`.

Run `python server/werkzeug_app.py` (or change the file to be executable first if you prefer). You should see the following in the terminal:

```
$ python server/werkzeug_app.py
# => WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
# => * Running on http://localhost:5555/ (Press CTRL+C to quit)
```

Go to `localhost:5555` and you should see the following:



A WSGI generated this response!

Back in the terminal, you should see a message generated from our request:

```
# => This web server is running at 127.0.0.1
# => 127.0.0.1 - - [29/Aug/2022 07:11:39] "GET / HTTP/1.1" 200 -
```

All of this together created an application and a web server that allowed us to access a message in our browser.

Conclusion

This was a brief introduction to web servers and WSGI. We won't typically build servers and applications in a single file, nor will we move requests and responses around without manipulating any of their attributes.

Most of the work performed by a WSGI will be invisible in modern web application frameworks, but it is important to know that this is going on behind the scenes. Without WSGIs, servers and Python applications would have a very difficult time communicating with one another. People might not be using Python for web development *at all* without its popular WSGIs.

We will explore more practical implementations of web servers with Flask and Werkzeug (and in greater depth) throughout Phase 4.

Solution Code

```
# server/werkzeug_app.py

from werkzeug.wrappers import Request, Response

@Request.application
def application(request):
    print(f'This web server is running at {request.remote_addr}')
    return Response('A WSGI generated this response!')

if __name__ == '__main__':
    from werkzeug.serving import run_simple
    run_simple(
        hostname='localhost',
        port=5555,
        application=application
```



Resources

- [Werkzeug - Pallets Projects](https://werkzeug.palletsprojects.com/en/2.2.x/quickstart/)  (<https://werkzeug.palletsprojects.com/en/2.2.x/quickstart/>)
- [What is a web server? - Mozilla](https://developer.mozilla.org/en-US/docs/Learn/Common_questions/What_is_a_web_server)  (https://developer.mozilla.org/en-US/docs/Learn/Common_questions/What_is_a_web_server)

This tool needs to be loaded in a new browser window

Load Web Servers and WSGI (CodeGrade) in a new window